МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра "ПОВТ и АС"

Технология Jakarta REST

Практикум

по выполнению лабораторной работы № 6

по дисциплине "Объектно-ориентированное программирование"

Ростов-на-Дону

20 г.

Составитель: к.ф.-м.н., доц. Габрельян Б.В.

Практикум по выполнению лабораторной работы по дисциплине "Объектно-ориентированное программирование". ДГТУ, г. Ростов на-Дону, 20 г.

В практикуме кратко изложены теоретические вопросы, необходимые для успешного выполнения лабораторной работы, рабочее задание и контрольные вопросы для самопроверки.

Предназначено для обучающихся по направлениям 090304 "Программная инженерия", 020303 "Математическое обеспечение и администрирование информационных систем".

Ответственный за выпуск:

зав. кафедрой Долгов В.В.

Издательский центр ДГТУ, 20

Оглавление

Введение ... 4

Теоретическая часть ... 4

1. Базовые концепции ... 4

2. Утилиты для работы с HTTP-запросами ... 6

3. Представления ресурсов ... 8

4. Технология Jakarta REST ... 9

4.1 Обзор API ... 10

4.2 Client API ... 22

Инструкция по технике безопасности при проведении работы ... 24

Цель работы ... 25

Рабочее задание ... 25

Общие положения ... 28

Материально-техническое обеспечение работы ... 28

Порядок выполнения работы ... 29

Контрольные вопросы ... 29

Перечень использованных информационных ресурсов ... 31

Введение

REST (REpresentational State Transfer – передача состояния представления) – это архитектурный стиль для быстрого создания, модификации и удаления ресурсов удаленно с использованием сетевых соединений. Сервер предоставляет ресурсы в ответ на запросы пользователей. Ресурсами является все, что клиент может получить от сервера: web-страницы, файлы, изображения, видео. Каждый ресурс имеет свой уникальный идентификатор (URI) и пользователь должен указать его в запросе. Один и тот же ресурс может быть представлен пользователю по-разному, например, как неформатированный текст или как текст с разметкой (xml-, JSON-документ, …) и т.п. В принципе REST не привязан к какому-то протоколу передачи данных по сети. На практике используется протокол HTTP (HTTPS). CRUD операции можно реализовать с помощью стандартных команд HTTP (GET, PUT, POST, DELETE). Конкретные реализации web-сервисов в стиле REST называются RESTfull сервисами. Технология разработки RESTfull сервисов в Jakarta EE называется Jakarta REST (старое название - JAX-RS) [1].

На сегодняшний день развитием технологии Jakarta REST занимается организация Eclipse Foundation, имена пакетов в этой технологии (как и в других технологиях Jakarta EE) начинаются с jakarta (в старых версиях с javax).

Теоретическая часть

1. Базовые концепции

Хорошая архитектура приложения предполагает, что оно разбито на части (модули, компоненты) и связи между отдельными частями являются по возможности слабыми. Тем не менее это связи, обеспечиваемые вызовами методов классов из разных модулей, пакетов внутри одного приложения.

Использование сервисов, в том числе web-сервисов позволяет сделать такие связи внешними по отношению к приложению. Отдельно создаются программные компоненты, реализующие web-сервисы и, отдельно, компоненты, запрашивающие их (являющиеся клиентами этих сервисов). Клиенты для получения сервиса посылают HTTP-запрос и получают результат в HTTP-ответе. В архитектуре REST сервисы, запрашиваемые и получаемые клиентом, рассматриваются как ресурсы. Таким образом, посылая запрос клиент должен указать идентификатор (URI) ресурса, команду поясняющую, что надо делать с ресурсом (GET, POST, …) и, быть может, данные, необходимые для работы с ресурсом, которые представляют собой тело передаваемого сообщения. На запрос клиент получает HTTP-ответ, тело которого может содержать запрашиваемый ресурс.

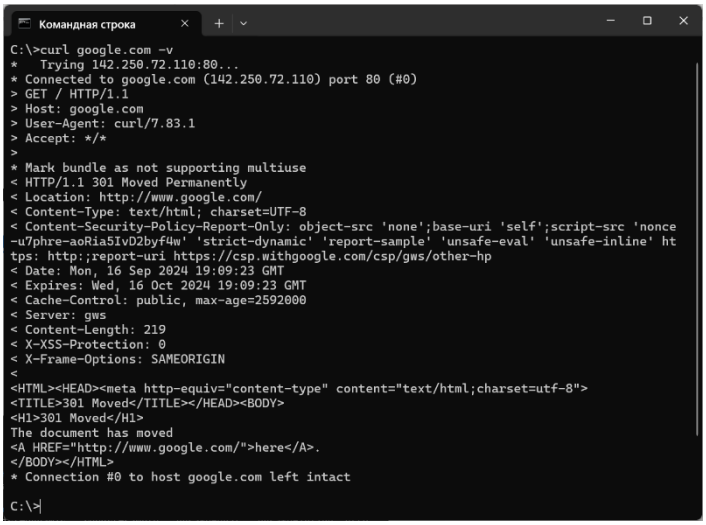
При использовании HTTP-протокола тело передаваемого и получаемого сообщения "заворачивается" в конверт, содержащий также заголовки, а для ответа еще и статус этого ответа.

Заголовки позволяют задавать различную информацию, характеризующую HTTP запрос или ответ. Это могут быть настройки локализации (язык, кодировка), формата (представления), в котором передается или возвращается тело запроса/ответа – неформатированный текст, текст с разметкой (html, xml, JSON, …), двоичные данные и т.п. Статус показывает был ли запрос обработан успешно или нет. В стандарте определены коды этого статуса – целые значения, состоящие из трех цифр. Первая цифра задает категорию сообщения: 1xx – информационные сообщения, 2xx – запрос обработан успешно, 3xx – произошло перенаправление запроса, 4xx – ошибка на стороне клиента, 5xx – ошибка на стороне сервера. В случае успешной

обработки GET и PUT запросы возвращают статус 200 (OK), POST запрос 201 (CREATED), DELETE запрос - статус 204 (NO CONTENT).

2. Утилиты для работы с HTTP-запросами

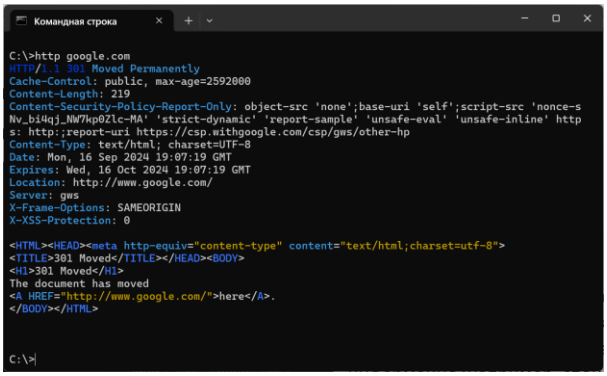
Клиентами созданного RESTfull сервиса может быть desktop-приложение, написанное на каком-то языке программирования, компоненты, размещенные в серверной части приложения, браузеры. Все браузеры поддерживают GET и POST запросы (адресная строка, форма на html-странице). Чтобы обрабатывать другие HTTP-команды нужно установить в браузере модуль расширения. Модули нужно подбирать под конкретный браузер. Например, одним из известных расширений для Google Chrome является RestMan, устанавливаемый (бесплатно) из магазина приложений Chrome Web Store.

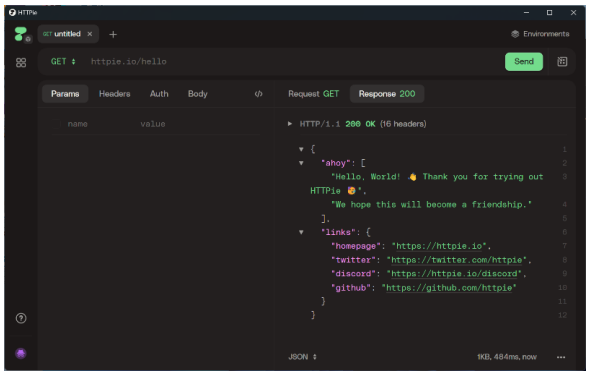


cURL – пример запроса

Но проверить работоспособность созданных сервисов можно и с помощью утилит, предназначенных для отправки и получения HTTP-запросов. Например, с помощью утилиты командной строки cURL, которую можно скачать с сайта https://curl.se или HTTPie, у которой есть версия с графическим пользовательским интерфейсом, а также версия для командной строки (сайт https://httpie.org/).

В дальнейших примерах будет использоваться cURL.





HTTPie в командной строке HTTPie GUI

cURL нужно передать как минимум адрес, по которому будет отослан запрос. В таком случае это будет GET запрос.

curl google.com

В результате отобразится только тело ответа (последние строки, начиная с <HTML> на рисунке "cURL – пример запроса" выше). Ключ –v (verbose) позволяет получить более подробную информацию (см. тот же рисунок). Если будет задан ключ –d (или --data) и следующий за ним некоторый текст, то будет выполнен POST запрос телом которого станет указанный текст.

curl localhost:8080/mylibrary **–d "<html><body>Hello</body></html>"**

Тип запроса можно, а для запросов, отличных от GET и POST необходимо задать явно с помощью ключа –X (или --request).

curl **–X PUT** localhost:8080/mylibrary –d "<html><body>Hello</body></html>"

Используя ключ –H (или --header) можно задать значение для какого-нибудь HTTP-заголовка (или каких-нибудь). Например, заголовок Accept определяет представление (формат) тела запроса, а заголовок Content-Type – представление тела ответа на запрос.

curl **–H "Accept: text/xml"** localhost:8080/mylibrary

curl **–H "Content-Type: text/xml"** localhost:8080/mylibrary –d "**<books><book><id>1</id><author>Mr.X</author><title>The Book</title></book></books>**"

3. Представления ресурсов

Представление, в котором будет отправлен или получен ресурс задается с помощью HTTP-заголовков (Content-Type, Accept). Представление text/plain задается в программе обычной строкой. Но если, например, указано, что ответ содержит http-страницу, должен быть отправлен текст (строка) с соответствующей разметкой ("<htm>…</html>"). Спецификация Jakarta REST поддерживает возможность преобразования данных из одного представления в другое. Например, для преобразования объектов наших классов в текстовое (без разметки или с xml-, html-, JSON- и т.п. разметкой) или байтовое представление. Для этого в общем случае нужно создать класс, обеспечивающий такое преобразование. Но, для некоторых, часто используемых представлений такие преобразования предоставляются конкретной реализацией технологии Jakarta REST. На сегодняшний день наиболее распространенным представлением для ресурсов является формат JSON. Преобразование выполняется с помощью стандартной технологии связывания объектов классов с JSON-файлами. Достаточно в HTTP-заголовке указать, что принимается или возвращается ресурс в JSON представлении, для того чтобы за сценой объект нашего класса был преобразован в JSON-текст или наоборот.

Для преобразования объектов Java классов в xml-документы и наоборот используется технология Jakarta XML binding (или, в старых версиях технологии JAX-B). Но ее поддержка на сегодняшний день не обеспечивается

автоматически. Минимальные требования таковы. Необходимо включить в файл pom.xml нужную зависимость (можно получить ее из центрального репозитория Maven https://mvnrepository.com/artifact/jakarta.xml.bind/jakarta.xml.bind-api).

```
<!-- https://mvnrepository.com/artifact/jakarta.xml.bind/jakarta.xml.bind-api -->
<dependency>
   <groupId>jakarta.xml.bind</groupId>
   <artifactId>jakarta.xml.bind-api</artifactId>
   <version>4.0.2</version>
</dependency>
```

Или скачать и добавить в проект и на сервер библиотеки jakarta.activation-api-2.1.3.jar и jakarta.xml.bind-api-4.0.2.jar. Можно, например, загрузить их одним файлом jar_files.zip с сайта https://jar-download.com/artifacts/jakarta.xml.bind. Кнопка Download jakarta.xml.bind-api.jar (4.0.2).

Кроме того, необходимо, как минимум аннотировать связываемый класс как @XmlRootElenment. Например,

```
@XmlRootElement
public class Book {
…
}
```

В простых случаях этого достаточно, чтобы объекты класса Book можно было передавать в запросах и получать из запросов в представлении text/xml.

4. Технология Jakarta REST

Спецификация Jakarta REST определяет набор API для разработки web-сервисов, построенных в соответствии с архитектурным стилем REST.

Технология основана на использовании POJO, HTTP-протокола, не зависит от формата, в котором представлено тело запроса, не привязана к конкретному web-серверу, является частью стека технологий Jakarta EE.

4.1 Обзор API

Аннотация @Path, заданная для обычного класса (POJO) объявляет его классом (корневого) ресурса REST и назначает ему URI. URI задается относительно контекста приложения.

```
@Path("/mylib")
public class MyBooks {
    …
}
```

Если имя приложения mylibrary (тогда по умолчанию и имя контекста mylibrary), то обращение к сервису в случае локально установленного сервера приложений Jakarta EE будет, например, таким

http://localhost:8080/mylibrary/mylib

Можно задать путь к приложению. Для этого в приложении нужно создать класс-наследник стандартного класса jakarta.ws.rs.core.Application и задать для него аннотацию jakarta.ws.rs.ApplicationPath. Например, задать путь "api" можно следующим образом

```
@ApplicationPath("/api")
public class MyLibsApplication extends Application {}
```

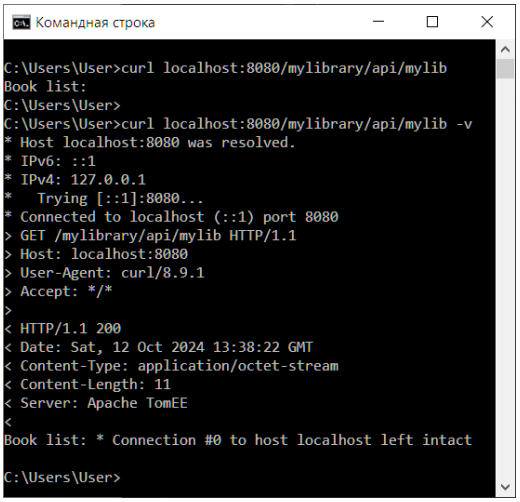
Тогда обращение к сервису будет выглядеть так

http://localhost:8080/mylibrary/**api**/mylib

Для обработки запросов пользователей класс ресурса должен предоставить методы, аннотированные в соответствии с тем, какую HTTP-команду обслуживает данный метод – GET, POST, …

Например, для ответов на GET запросы в классе ресурса должен быть метод с аннотацией @GET

```
@Path("/mylib")
public class MyBooks {
    @GET
    public String getBooks() {
        return "Book list: ";
    }
}
```



Методы класса-ресурса должны возвращать (или получать) ресурс в правильном представлении. Например, если нужно вернуть html-страницу, то в примере выше в возвращаемой строке нужно задать правильную разметку, скажем,

```
@GET
public String getBooks() {
    return "<html><body><p>Book list: </p></body></html>";
}
```

Методы могут возвращать не только строки, но и объекты классов, если для них определено преобразование в требуемое представление. Например, если есть класс

```
// аннотация @XmlRootElenment объявлена в пакете jakarta.xml.bind.annotation
@XmlRootElement
public class Book {
   private long id;
   private String title;
```

```
    private String author;
    public Book(long id, String title, String author) {
        this.id = id;    this.title = title;      this.author = author;
    }
    public Book() { this(-1, "","");  }
    public long getId() { return id; }
    public void setId(long id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
    public void setAuthor(String author) { this.author = author; }
}
```

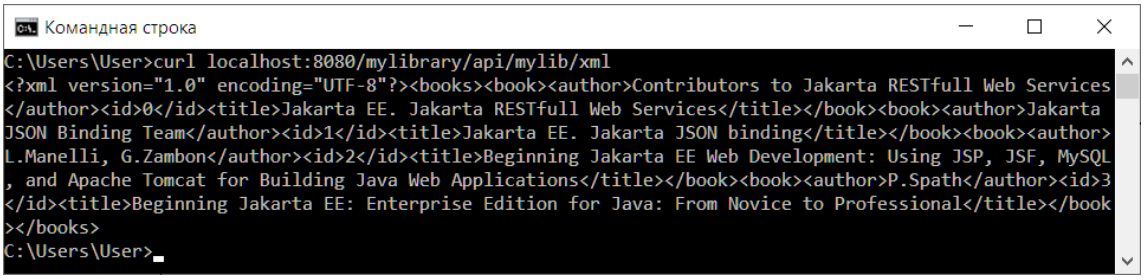
То его объекты можно возвращать, если в запросе пользователя указано представление в виде JSON-файла или (благодаря @XmlRootElement) в виде xml-файла, так как Jakarta REST поддерживает технологии JSON- и XML-связывания. Для этого метод, возвращающий ресурс должен быть аннотирован **@Produces** с указанием соответствующего MIME-типа в виде строки, например, "text/xml", "application/json" или как значение перечислимого типа jakarta.ws.rs.core.MediaType, например, MediaType.TEXT_XML, MediaType.APPLICATION_JSON. Метод (например, POST), получающий из запроса данные в указанном представлении, должен быть аннотирован **@Consumes** с соответствующим MIME–типом.

Аннотация Path, указанная не для класса в целом, но для отдельного метода определяет подресурс и задает путь к нему относительно корневого ресурса.

```
…
@Path("/mylib")
```

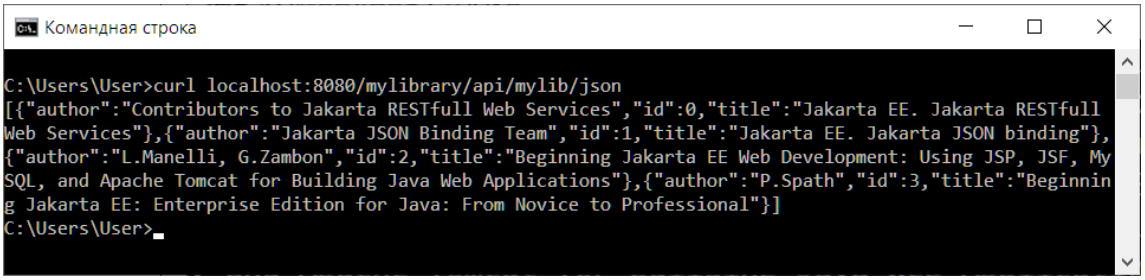
```
public class MyBooks {
	private List<Book> books = List.of(
		new Book(0,"Jakarta EE. Jakarta RESTfull Web Services", "Contributors to Jakarta RESTfull Web Services"),
		new Book(1,"Jakarta EE. Jakarta JSON binding", "Jakarta JSON Binding Team"),
		new Book(2,"Beginning Jakarta EE Web Development: Using JSP, JSF, MySQL, and Apache Tomcat for Building Java Web Applications", "L.Manelli, G.Zambon"),
		new Book(3,"Beginning Jakarta EE: Enterprise Edition for Java: From Novice to Professional", "P.Spath") );
	@GET
	@Path("/xml")
	@Produces("text/xml")
	public List<Book> getBooksXML() { return books; }

	@GET
	@Path("/json")
	@Produces("application/json")
	public List<Book> getBooksJSON() { return books; }
}
```

Тогда для получения xml-представления нужно выполнить запрос localhost:8080/mylibrary/api/mylib/xml



А для получения JSON-представления запрос

localhost:8080/mylibrary/api/mylib/json



Видно, что в данном случае методы getBooksXML() и getBooksJSON() выполняют одни и те же действия. Можно создать один метод, например, getBooks() и указать, что он может генерировать разные представления, тогда конкретное представление будет определяться настройками запроса (для GET - значением http-заголовка Accept).

Например, изменяя класс MyBooks следующим образом

```
…
@GET
   @Path("/format")
   @Produces({"application/json", "text/xml"})
   public List<Book> getBooks() {
      return books;
   }
…
```

можно выполнить запрос localhost:8080/mylibrary/api/mylib/json со значением http-заголовка Accept text/xml или application/json.

```
Командная строка
C:\Users\User>curl -H "Accept: text/xml" localhost:8080/mylibrary/api/mylib/format
<?xml version="1.0" encoding="UTF-8"?><books><book><author>Contributors to Jakarta RESTfull Web Services
</author><id>0</id><title>Jakarta EE. Jakarta RESTfull Web Services</title></book><book><author>Jakarta
JSON Binding Team</author><id>1</id><title>Jakarta EE. Jakarta JSON binding</title></book><book><author>
L.Manelli, G.Zambon</author><id>2</id><title>Beginning Jakarta EE Web Development: Using JSP, JSF, MySQL
, and Apache Tomcat for Building Java Web Applications</title></book><book><author>P.Spath</author><id>3
</id><title>Beginning Jakarta EE: Enterprise Edition for Java: From Novice to Professional</title></book
></books>
C:\Users\User>
C:\Users\User>curl -H "Accept: application/json" localhost:8080/mylibrary/api/mylib/format
[{"author":"Contributors to Jakarta RESTfull Web Services","id":0,"title":"Jakarta EE. Jakarta RESTfull
Web Services"},{"author":"Jakarta JSON Binding Team","id":1,"title":"Jakarta EE. Jakarta JSON binding"},
{"author":"L.Manelli, G.Zambon","id":2,"title":"Beginning Jakarta EE Web Development: Using JSP, JSF, My
SQL, and Apache Tomcat for Building Java Web Applications"},{"author":"P.Spath","id":3,"title":"Beginnin
g Jakarta EE: Enterprise Edition for Java: From Novice to Professional"}]
C:\Users\User>
```

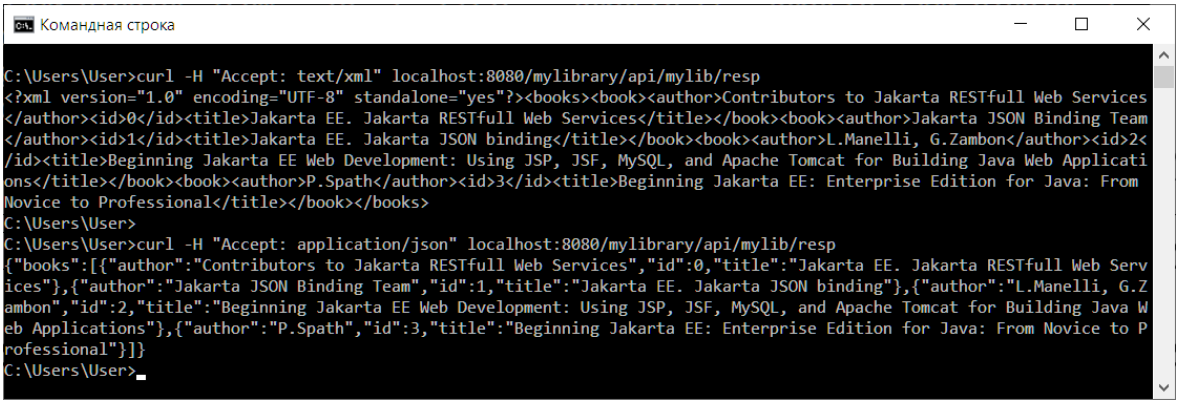
В спецификации Jakarta REST определен также класс (абстрактный) Response и вложенный класс Response.ResponseBuilder. Эти классы (точнее их конкретные реализации) позволяют методу класса-ресурса вернуть помимо собственно тела ответа, также его http-заголовки и код статуса. Телом ответа должен быть объект класса. Если, как в примере выше, метод (getBooks()) возвращает не отдельный объект, но коллекцию объектов, то для возврата представления в виде JSON-фала можно ничего не менять, но для xml-представления нам нужно создать дополнительный класс, который будет представлять коллекцию. Например,

```
…
import jakarta.xml.bind.annotation.XmlElement;
import jakarta.xml.bind.annotation.XmlRootElement;
import java.util.List;
…
@XmlRootElement
public class Books {
   private List<Book> books;
```

```
    public List<Book> getBooks() {
        return books;
    }
    @XmlElement(name = "book") /* внутри корневого тега <books> будут тэги
<book> */
    public void setBooks(List<Book> blist) {
        books = blist;
    }
}
```

Тогда (и для JSON-, и для XML-представления) метод, обрабатывающий GET-запрос может возвращать Response.

```
@Path("/mylib")
public class MyBooks {
…
@GET
    @Path("/resp")
    @Produces({"application/json", "text/xml"})
    public Response getBooksResp() {
        Books blist = new Books();
        blist.setBooks(books);
        return Response.ok().entity(blist).build();
    }
…
```



Все пути к ресурсам в примерах выше являются статическими, в них нет элементов, способных меняться от запроса к запросу. Что если мы хотим дать пользователю возможность получать по запросу информацию не обо всех книгах, а только о какой-то одной, например, с id равным 1, или 2, … Тогда в запросе должна содержаться изменяющаяся (динамическая) информация - 1, 2, ... Как обрабатывать такие запросы одним методом класса-ресурса?

Есть несколько возможностей.

Во-первых, такую информацию можно передавать через параметры запроса

GET localhost:8080/mylibrary/api/mylib/fmt**&id=1**

GET localhost:8080/mylibrary/api/mylib/fmt**&id=2**

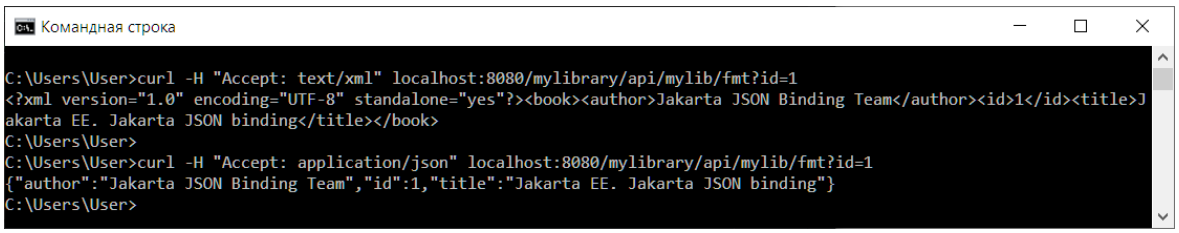
…

Параметры можно передавать не только в GET- но и в других (POST-, PUT-, …) запросах.

Получить значение параметра запроса в методе можно с помощью аннотации **@QueryParam**, заданной для аргумента соответствующего метода.

```
…
@Path("/mylib")
public class MyBooks {
…
   @GET
```

```
    @Path("/fmt")
    @Produces({"application/json", "text/xml"})
    public Book getBook(@QueryParam("id") int id) {
        if(id<0 || id>books.size()-1) return new Book();
        return books.get(id);
    }
}
…
```



Если в запросе передается несколько параметров, у соответствующего метода будет несколько аргументов, аннотированных @QueryParam.

Во-вторых, в случае, если обращение к ресурсу выполняется с формы на html-странице, получить значения управляющих элементов формы можно с помощью аннотации **@FormParam** (просто заменить ею аннотацию @QueryParam в примере выше).

В-третьих, динамическая информация может передаваться через URI (строку запроса) как часть пути, то есть как параметр пути

GET localhost:8080/mylibrary/api/mylib/fmt/**1**

GET localhost:8080/mylibrary/api/mylib/fmt/**2**

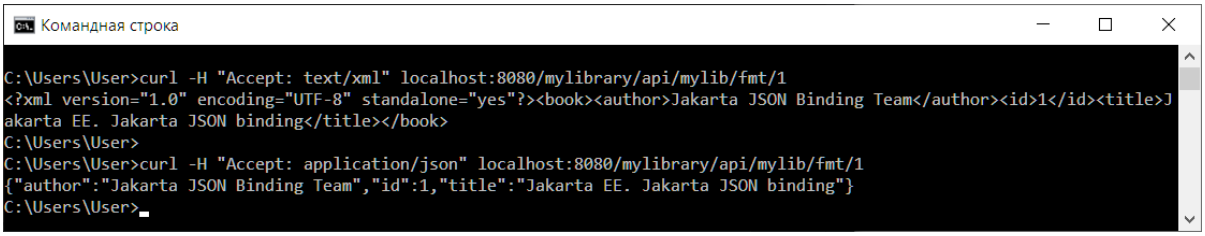
…

Для получения значения параметра пути в методе нужно в аннотации @Path использовать шаблон в виде **{имя}** и аннотацию **@PathParam**.

…

```
@Path("/mylib")
```

```
public class MyBooks {
…
   @GET
   @Path("/fmt/{id}")
   @Produces({"application/json", "text/xml"})
   public Book getBookById(@PathParam("id") int i) {
      if(i<0 || i>books.size()-1) return new Book();
      return books.get(i);
   }
…
```



Для добавления (создания) нового ресурса используется POST-запрос. В нем помимо URI должны также содержаться добавляемые данные в правильном представлении (HTTP-заголовок Content-type). Аннотация **@Consumes** позволяет указать, какое представление ожидает получить метод, обрабатывающий запрос, а через свой аргумент метод может получить переданные в запросе данные.

```
…
@Path("/mylib")
public class MyBooks {
   private static List<Book> books = new ArrayList<>();
   static {
         books.add(new Book(0,"Jakarta EE. Jakarta RESTfull Web Services", "Contributors to Jakarta RESTfull Web Services"));
```

books.add(new Book(1,"Jakarta EE. Jakarta JSON binding", "Jakarta JSON Binding Team"));

books.add(new Book(2,"Beginning Jakarta EE Web Development: Using JSP, JSF, MySQL, and Apache Tomcat for Building Java Web Applications", "L.Manelli, G.Zambon"));

books.add(new Book(3,"Beginning Jakarta EE: Enterprise Edition for Java: From Novice to Professional", "P.Spath"));

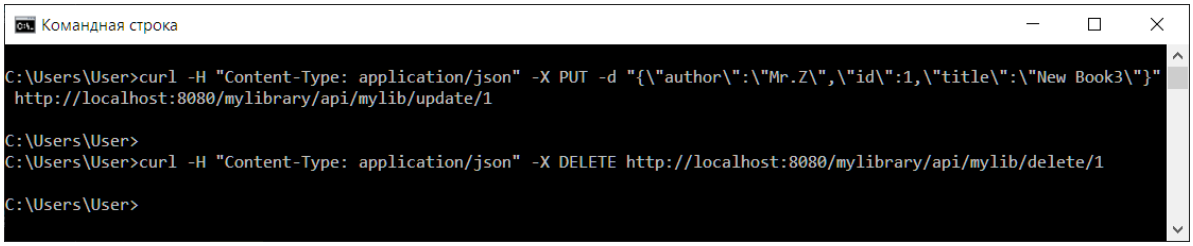
```
    }
…
    @POST
    @Path("/create")
    @Consumes({"application/json", "text/xml"})
    public void createBook(Book b) {
        books.add(b);
    }
…
```

Изменение ресурса выполняется в PUT–запросе, а его удаление – в DELETE-запросе.

```
…
@Path("/mylib")
public class MyBooks {
…
```

```
@PUT
@Path("/update/{id}")
public void updateBook(@PathParam("id") int id, Book book) {
  // проверка на правильный id
  …
  int index = 0;
  // найти индекс для заданного id
  …
  books.get(index).setAuthor(book.getAuthor());
  books.get(index).setTitle(book.getTitle());
}
@DELETE
@Path("/delete/{id}")
public void delete(@PathParam("id") int id) {
   // проверка на правильный id
  …
  int index = 0;
  // найти индекс для заданного id
  …
   books.remove(index);
}
```

…



4.2 Client API

Jakarta REST содержит специальный программный интерфейс для выполнения клиентских запросов из Java программ к RESTfull сервисам – Client API. Этот API определен в пакете jakarta.ws.rs.client (для старых версий javax.ws.rs.client) и может использоваться как в Java SE, так и в web- и корпоративных приложениях. Основные классы – ClientBuilder, Client, WebTarget (цель ресурса с его URI), Entity (представляет объект сообщения). Прежде всего нужно получить ссылку на объект класса Client.

```
Client client = ClientBuilder.newClient();
```

Затем задать URI запрашиваемого ресурса (может быть задан не полный путь к ресурсу, далее можно будет дополнить этот путь перед отправкой запроса).

```
int id = 1;
WebTarget target =
client.target("localhost:8080/mylibrary/api/mylib/fmt/"+id);
```

Через ссылку на WebTarget можно выполнить запрос и получить отклик на него, например,

```
Response resp = target.request().get();
// String s = target.request().get(String.class);
…
resp.close();
```

Можно задать параметры, передаваемые в адресной строке запроса, например, чтобы послать запрос http://localfost:8080/mylibrary/api/mylib/fmt?id=1

```
int id = 1;
WebTarget target =
client.target("localhost:8080/mylibrary/api/mylib/fmt/");
Response resp = target.queryParam("id", id).request().get();
```

Для POST и PUT запросов необходимо передать тело запроса, например, объект класса Book, завернув его в объект Entity.

```
Book book = new Book();
Response resp = target.request().post(Entity.entity(book));
```

В классе Entity предусмотрены методы (статические) для преобразования объекта класса в одно из предопределенных представлений:

```
Entity.text(book);
Entity.html(book);
Entity.xhml(book);
Entity.xml(book);
Entity.json(book);
```

Пример класса, поддерживающего клиентские запросы к RESTfull web-сервису на выполнение основных CRUD операций.

```
…
import jakarta.ws.rs.client.*;
…
public class BooksClient {
    private Client client = ClientBuilder.newClient();
    private WebTarget target =
        client.target("http://localhost:8080/mylibrary/api /mylib");
…
    public Book[] getAll() {
        Book[] books =
      target.path("format").request("application/json").get(Book[].class);
        return books;
    }
```

```
public Book get(long id) {
        Book book =
        target.path("fmt/"+id).request("application/json").get(Book.class);
        return book;
}
public void add(Book b) {
    target.path("create").request("application/json").post(Entity.json(b),
Book.class);
// вместо Entity.json(b) можно Entity.entity(b)
}
public void update(Book b) {
        target.post("update/"+b.getId()).request("application/json").put(
        Entity.json(b), Book.class);
}
public void delete(long id) {
        target.path("/"+id).request("application/json").delete( Book.class);
}
…
}
```

Инструкция по технике безопасности при проведении работы

При выполнении работы нужно выполнять требования по технике безопасности при работе с электронными приборами.

Цель работы

Изучение технологии Jakarta REST (JAX-RS) – создание web-сервисов в стиле REST в корпоративных приложениях Java. Получение навыков проектирования и реализации web-приложений Java, использующих web-сервисы REST.

Рабочее задание

ЗАДАНИЕ 1. На сервере хранится список пользователей. По каждому пользователю хранится следующая информация: имя, фамилия, адрес электронной почты, пароль. По технологии JAX-RS создать RESTfull сервис для добавления нового пользователя.

ЗАДАНИЕ 2. Проверить на работоспособность полученный в предыдущем задании сервис с помощью клиентского web-приложения, с формой (<form method="POST"> … </form>), через которую посылается запрос к сервису. По умолчанию MIME-тип содержимого запроса (атрибут enctype тега form) – application/x-www-form-urlencoded.

ЗАДАНИЕ 3. На сервере хранится список пользователей. По каждому пользователю хранится следующая информация: имя, фамилия, адрес электронной почты, пароль. По технологии JAX-RS создать RESTfull сервис для получения информации о пользователе по адресу его электронной почты, получения списка всех пользователей.

ЗАДАНИЕ 4. Проверить на работоспособность полученный в предыдущем задании сервис с помощью клиентского web-приложения, с формой (<form method="GET">…</form>), через которую посылается запрос к сервису.

ЗАДАНИЕ 5. На сервере хранится список пользователей. По каждому пользователю хранится следующая информация: имя, фамилия, адрес электронной почты, пароль. По технологии JAX-RS создать RESTfull сервисы для изменения и удаления информации о пользователе с заданным значением адреса электронной почты.

ЗАДАНИЕ 6. Проверить на работоспособность полученные в предыдущих заданиях (1, 3, 5) сервисы с помощью cURL, HTTPie или модулей расширения браузера.

ЗАДАНИЕ 7. На сервере хранится информация о некотором соревновании: название, дата и время проведения, организаторы, доп. информация (например, спонсоры), названия отдельных этапов (например, открытие, 1/16 финала, 1/8 финала, 1/4 финала, полуфинал, финал, закрытие) и время их проведения, списки участников с их результатами.

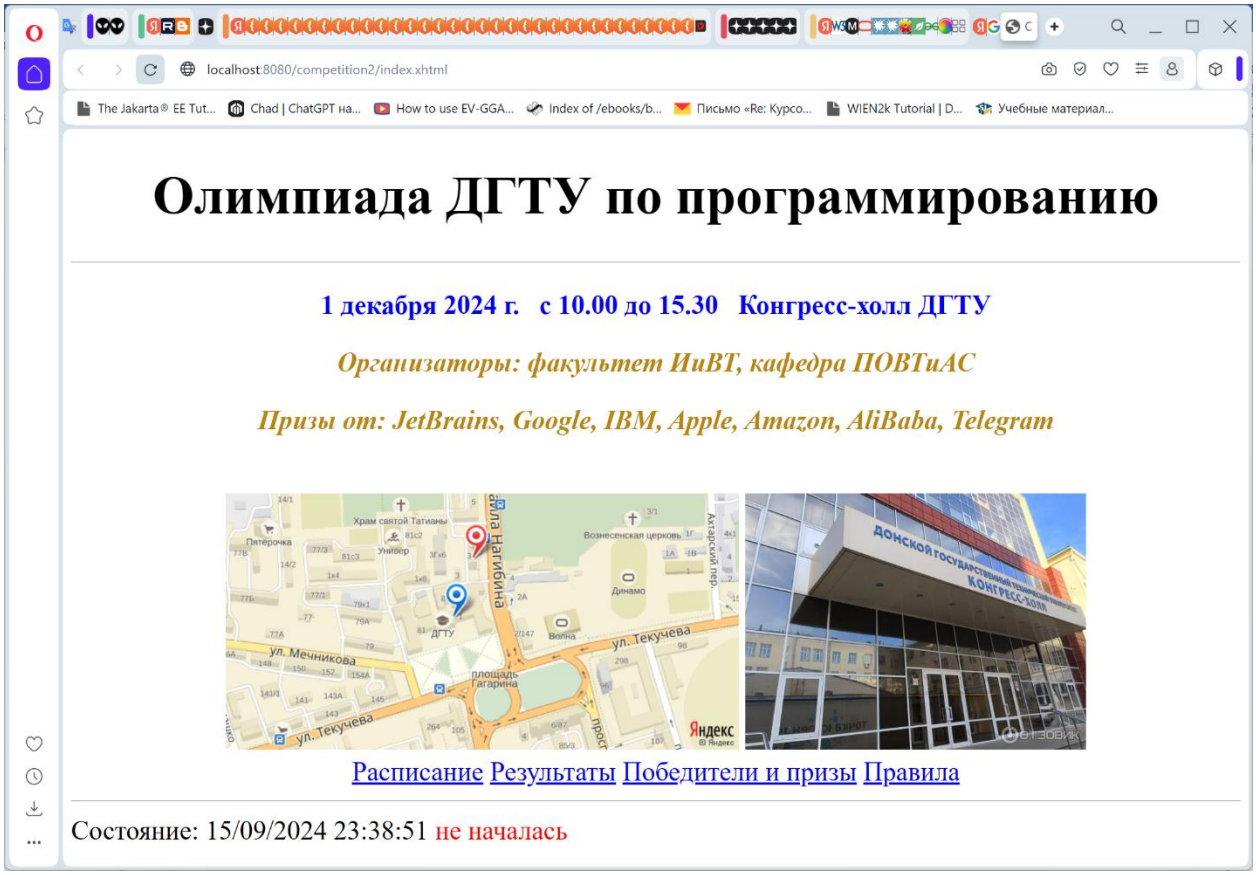
Реализовать RESTfull сервисы по технологии JAX-RS, позволяющие по соответствующим HTTP-запросам получать, добавлять, изменять удалять хранящуюся на сервере информацию. Данные в теле запроса и в теле ответа должны быть представлены в виде JSON-документов.

ЗАДАНИЕ 8. Проверить на работоспособность полученные в предыдущем задании сервисы с помощью cURL, HTTPie или модулей расширения браузера.

ЗАДАНИЕ 9. Программа для организаторов различных соревнований (олимпиады, конкурсы, спортивные турниры, …). Три вида пользователей: администратор – задает тему соревнования, расписание проведения соревнования, вносит отдельные промежуточные результаты; участник – регистрируется для участия в соревновании и в заданные сроки проходит

индивидуальные этапы соревнования; зритель – не регистрируется, может просматривать промежуточные и окончательные результаты соревнования.

Стартовая страница созданного администратором приложения может выглядеть похоже на



Часть программы, отвечающая за информационную составляющую – хранение данных о соревновании, обработка запросов на получение информации, внесение новой информации, изменение существующей (в результате проведения промежуточных этапов), удаление информации должно быть организовано в виде RESTfull сервисов, на основе спецификации JAX-RS, а обращение к сервисам – с помощью Client API.

Общие положения

Технология Jakarta REST (JAX-RS) определяет API для создания и использования RESTfull сервисов в корпоративных и web-приложениях на Java и является также ключевой технологией, используемой при создании микросервисов и облачных приложений. Заявленный в спецификации Jakarta REST API позволяет создавать web–сервисы с помощью небольшого количества аннотаций, применяемых к обычным объектам Java (POJO). Аннотации дают возможность решать различные задачи: создание REST-ответов на запросы пользователей, указание представления для данных ответа, выполнение стандартных операций над данными – создание, чтение, модификация, удаление. Кроме того, спецификация предлагает стандартный API для создания клиентских приложений, запрашивающих REST-сервисы.

Материально-техническое обеспечение работы

Компьютеры с установленным программным обеспечением:

1. Java SDK 11+.

2. IDE для работы с корпоративными Java-проектами - желательно IntelliJ Idea Enterprise.

3. Сервер приложений с поддержкой Java EE/Jakarta EE - желательно Apache TomEE+ 8+.

4. Сервер реляционных баз данных (СУБД), например, PostgreSQL (опционально).

5. ConnectorJ - драйвер для работы с сервером реляционных баз данных (например, PostgreSQL) из Java-программ (опционально).

Порядок выполнения работы

1. Ознакомиться с теоретическим материалом.

2. Выполнять задания в порядке их следования так как при выполнении текущего задания потребуются результаты, полученные при выполнении предыдущих заданий.

Контрольные вопросы

1. Что такое Jakarta REST (JAX-RS)?
2. Как задается путь к ресурсу?
3. Что в программе можно аннотировать как @Path?
4. Какая HTTP-команда используется для получения ресурса?
5. Какая HTTP-команда используется для добавления ресурса?
6. Какая HTTP-команда используется для изменения ресурса?
7. Какая HTTP-команда используется для удаления ресурса?
8. Для чего используется аннотация @Produces?
9. Для чего используется аннотация @Consumes?
10. Какой HTTP-заголовок задает тип передаваемого в запросе содержимого?
11. Какой HTTP-заголовок задает тип возвращаемого по запросу содержимого?
12. Что такое параметр командной строки запроса (параметр запроса)?
13. Какая аннотация позволяет получить значение параметра командной строки запроса?
14. Какая аннотация позволяет получить значение параметра из параметризованного пути запроса?
15. Как задается параметризованный путь к ресурсу?
16. Что такое представление ресурса?

17. Что задает аннотация @ApplicationPath?

18. Как создать подресурс?

19. Для каких представлений ресурсов в технологии Jakarta REST имеются стандартные преобразования?

20. Что такое Client API?

Перечень использованных информационных ресурсов

1. Jakarta EE. Jakarta RESTfull Web Services. Contributors to Jakarta RESTfull Web Services. https://github.com/jakartaee/rest/graphs/contributors. Version 4.0.0. – 2024, 101 p.
2. Jakarta EE. Jakarta JSON binding. Jakarta JSON Binding Team, https://projects.eclipse.org/projects/ee4j.jsonb. – 2022, 35 p.
3. Э.Гонсалвес "Изучаем Java EE 7". – СПб: Питер. – 2016, 640с.
4. Д.Хеффельфингер "Java EE 6 и сервер приложений GlassFish 3". – М.:ДМК-Пресс. – 2013, 416 с.
5. А.Гупта "Java EE 7. Основы". – М.:Вильямс. – 2014, 336 с.
6. L.Manelli, G.Zambon "Beginning Jakarta EE Web Development: Using JSP, JSF, MySQL, and Apache Tomcat for Building Java Web Applications". – Apress. – 2020, 416 с.
7. P.Spath "Beginning Jakarta EE: Enterprise Edition for Java: From Novice to Professional", 2-d ed. – Apress. – 2019, 454с.

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 120. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344010, г. Ростов-на-Дону, пл. Гагарина, 1.